

Yawn State Detection Using Deep Learning

1. Introduction

This project is part of a broader Driver Drowsiness Detection System, which aims to monitor visual cues such as eye openness and yawning to assess driver alertness. The complete system is divided into two components: (1) detecting whether the driver's eyes are open or closed, and (2) detecting yawns based on mouth shape. My individual contribution to the Driver Drowsiness Detection System, focused exclusively on yawn detection using deep learning. While the broader system analyzes both eye and mouth-based cues to detect fatigue, my work specifically addresses the challenge of identifying yawning behavior from facial images. Yawning is a critical early indicator of drowsiness. Accurately detecting it in real time can help issue timely fatigue alerts, potentially preventing road accidents. The goal of this project was to build, compare, and interpret deep learning models capable of classifying images into two categories: 'yawn' and 'non-yawn'.

To support this task, I used a labeled dataset from Kaggle [1] consisting of real-world driver face images. The dataset includes:

Training set: 617 'yawn' and 616 'non-yawn' images

Test set: 106 'yawn' and 109 'non-yawn' images

I developed and trained two models:

A custom Convolutional Neural Network (CNN) built from scratch using PyTorch

A ResNet18 model pretrained on ImageNet and fine-tuned for this task

Both models were trained using data augmentation and evaluated on classification metrics including precision, recall, and macro F1-score. In addition, I applied Grad-CAM (Gradient-weighted Class Activation Mapping) to visualize which parts of the image contributed most to each model's predictions. This was crucial to confirm that both models were attending to meaningful facial features—particularly the mouth region for yawn detection.

2. Description

To perform yawn detection, I developed two deep learning models: a custom Convolutional Neural Network (CNN) and a ResNet18-based model. Both were trained to classify facial images into two categories: 'yawn' and 'non-yawn'.

Dataset & Preprocessing - The dataset was sourced from Kaggle [1], structured in directory format with subfolders for each class. After filtering and cleaning, the dataset included. All images were resized to 100×100 pixels for the custom CNN and 224×224 pixels for ResNet18 (to match its input requirements). Images were converted to RGB format and normalized to [0, 1] during preprocessing.

Custom CNN Architecture- The custom CNN was implemented from scratch using PyTorch and included:

3 convolutional blocks, each consisting of:

Conv2D → BatchNorm → GELU Activation → MaxPooling

Dropout layers after each block to reduce overfitting

A Global Average Pooling layer to flatten feature maps

A fully connected (Linear) classifier head with:

Linear → GELU → Dropout → Linear (output: 2 classes)

This architecture was designed to be lightweight and fast while still extracting meaningful features from the input images.

ResNet18 Architecture

To benchmark against a stronger backbone, I fine-tuned a ResNet18 model pretrained on ImageNet. I modified the classifier head to output 2 logits (for binary classification: yawn/non-yawn) and replaced the final activation with a softmax during evaluation.

Training Details:

Both models were trained using:

Loss Function: CrossEntropyLoss (since the task is binary classification)

Optimizer: Adam with learning rate 0.001

Data Augmentation: Applied via torchvision.transforms:

- Random horizontal flips

- Random rotations (± 15 degrees)

- Color jitter (brightness and contrast adjustments)

- Batch Size: 32

- Epochs: 15

Evaluation Metrics

For both models, I tracked the following metrics on the test set:

- Precision

- Recall

- F1-score (macro average)

- Accuracy

Grad-CAM Explain ability-

After training, I applied Grad-CAM (Gradient-weighted Class Activation Mapping) to both models to visualize which parts of the image influenced predictions. For the custom CNN, Grad-CAM was applied to the last convolutional layer; for ResNet18, it was applied to the final residual block. The generated heatmaps highlighted the mouth area in most cases, confirming that both models were learning task-relevant features.

3. Individual Contribution

For this project, my role focused entirely on developing and evaluating the yawn detection component of the Driver Drowsiness Detection System. My key responsibilities covered everything from data handling to model design, training, evaluation, and explainability. Here's a breakdown of my contributions:

Data Preparation:

I prepared the dataset by loading facial images from directory-based folders, ensuring a clean split between 'yawn' and 'non-yawn' classes. I resized the images to

fit both the custom CNN (100×100) and ResNet18 (224×224) input requirements and applied normalization to standardize inputs.

Custom CNN Implementation:

I designed and built a lightweight CNN architecture from scratch using PyTorch. The model includes 3 convolutional blocks with BatchNorm, GELU activations, and Dropout layers for regularization. A global average pooling layer flattens the feature maps before classification. I optimized the architecture for simplicity and real-time readiness.

ResNet18 Benchmarking:

To provide a stronger baseline, I fine-tuned a ResNet18 model pretrained on ImageNet. I customized the output head to handle binary classification and ensured the training setup was aligned with the custom CNN for fair comparison.

Data Augmentation:

I integrated data augmentation techniques (random flips, rotations, and color jitter) to make the models more robust to variations in lighting and pose—critical for real-world applications.

Training & Evaluation :I wrote and tested the full training pipeline, incorporating:
Adam optimizer

CrossEntropyLoss

Best model checkpointing based on macro F1-score

I evaluated both models on precision, recall, accuracy, and macro F1 to assess their performance.

Explainability with Grad-CAM: I applied Grad-CAM to both models to visualize which regions of the image influenced their predictions. This was essential for verifying that the models were truly focusing on the mouth area—the most important feature for yawn detection.

Result Analysis & Comparison:

I analyzed the classification results and compared the performance of my custom CNN with ResNet18, both quantitatively (metrics) and qualitatively (Grad-CAM heatmaps).

4. Results

I trained and evaluated both the custom CNN and the ResNet18 models on the yawn detection task using the test set of 106 'yawn' and 109 'non-yawn' images. The evaluation focused on key classification metrics: precision, recall, F1-score, and accuracy.

Custom CNN Performance

The custom CNN achieved solid performance, but it did make some classification errors, particularly with yawn detection. Here's the confusion matrix and metric summary:

	Predicted No Yawn	Predicted Yawn
Actual No Yawn	101	8
Actual Yawn	18	88

Accuracy: 0.88

Macro F1-Score: 0.87

Class	Precision	Recall	F1-Score
Yawn	0.92	0.83	0.87
Non-Yawn	0.85	0.93	0.89

Takeaways: The CNN was strong at detecting non-yawns (93% recall) but missed some yawns (83% recall), leading to a few false negatives.

Overall, it maintained balanced precision and recall but fell short of perfect detection.

ResNet18 Performance: The ResNet18 model excelled, achieving near-perfect results. Here's the confusion matrix:

	Predicted No Yawn	Predicted Yawn
Actual No Yawn	109	0
Actual Yawn	0	106

Accuracy: 1.00

Macro F1-Score: 1.00

Class	Precision	Recall	F1-Score
Yawn	1.00	1.00	1.00
Non-Yawn	1.00	1.00	1.00

Takeaways:

The ResNet18 model predicted all yawning cases correctly (100% recall) and made no false predictions at all.

It achieved perfect precision and recall across both classes, confirming its high reliability.

5. Explain ability with Grad-CAM

To better understand how the models make predictions and ensure that their decisions are based on meaningful facial features (specifically, the mouth area for yawning), I applied Grad-CAM (Gradient-weighted Class Activation Mapping) to both the custom CNN and the ResNet18 models.

Grad-CAM generates heatmaps that highlight the regions of the image most responsible for a model's prediction. This technique is especially useful for

confirming whether the models are focusing on the correct visual cues—in this case, the driver’s mouth.

Custom CNN Grad-CAM Results

The Grad-CAM visualizations for the custom CNN showed inconsistent attention: In some cases, the model correctly focused on the mouth area, especially when the yawns were large and obvious.

However, in other cases, the heatmaps were scattered and noisy, showing attention spread across unrelated facial regions like the cheeks, jawline, or even the nose.

This lack of consistency suggests that while the custom CNN could learn some relevant patterns, it sometimes relied on irrelevant features, which may explain its occasional misclassifications.

ResNet18 Grad-CAM Results

The Grad-CAM visualizations for ResNet18 were much sharper and more reliable: The heatmaps consistently focused on the mouth region, even when the yawn was subtle or partially obstructed.

The model ignored irrelevant areas (such as eyes or background), demonstrating that it had learned task-specific features effectively.

This precise focus is likely a major factor behind ResNet18’s perfect test performance, as it reliably identified the key indicators of yawning.

6 Summary and Conclusions - In this report, I presented my individual contribution to the Driver Drowsiness Detection System, focusing specifically on yawn detection using deep learning. The objective was to develop models that can reliably classify facial images as either ‘yawn’ or ‘non-yawn,’ providing an early-warning mechanism for driver fatigue.

I developed and compared two models:

 A custom CNN built from scratch

 A ResNet18 model fine-tuned from pretrained weights

Both models were trained on a labeled dataset (617 yawns and 616 non-yawns for training; 106 yawns and 109 non-yawns for testing) and evaluated using precision, recall, and F1-score.

Key Results	Model	Precision (Yawn)	Recall (Yawn)	F1-Score (Yawn)
Macro F1-Score				
Custom CNN	0.92	0.83	0.87	0.87
ResNet18	1.00	1.00	1.00	1.00

The custom CNN performed well overall but missed some yawns, likely due to limited complexity in its architecture. The ResNet18 achieved perfect performance on the test set, correctly identifying all yawns and non-yawns.

Explain ability Insights

Using Grad-CAM: The custom CNN showed mixed attention, sometimes focusing on irrelevant facial areas. The ResNet18 consistently focused on the mouth region, validating its strong performance.

Conclusion-- This work demonstrated that while a lightweight custom CNN can achieve respectable results, a deeper, pretrained model like ResNet18 is significantly more effective for yawn detection. The combination of high performance and reliable explain ability makes ResNet18 a better candidate for real-world deployment in driver monitoring systems.

Future Work :

Expand the dataset with more diverse faces and angles to improve robustness.

Integrate the model into a real-time webcam pipeline for live monitoring.

Explore temporal modeling (e.g., LSTM + CNN) to capture yawning sequences rather than static images.

7. Code Reuse Estimate

For this project, I developed the yawn detection pipeline mostly from scratch, ensuring the code was modular, clean, and reusable for future real-time integration. Here's an estimate of code reuse:

Aspect	Lines of Code
Total lines in final scripts	~400
Lines adapted from external sources	~20
Reuse Percentage	$(20 / 400) \times 100 \approx 5\%$

Most of the reused code consisted of standard PyTorch components (e.g., model training loops, Grad-CAM base functions) that were slightly modified to fit the needs of my custom CNN and ResNet18 implementation. The majority of the architecture design, augmentation pipeline, evaluation logic, and visualization functions were written manually and tailored for the specific requirements of yawn detection.

8. References

🔗 **Kaggle Dataset:**

Serena Raju, Abdelaziz Nebil, “Yawn Eye Dataset (New)”,
<https://www.kaggle.com/datasets/serenaraju/yawn-eye-dataset-new>

🔗 **PyTorch Documentation:**

<https://pytorch.org/docs/stable/index.html>

🔗 **Scikit-learn Metrics Documentation:**

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

🔗 **Grad-CAM Paper:**

Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization*. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 618-626.

🔗 **ResNet Paper:**

He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.

🔗 **GELU Activation Function:**

Hendrycks, D., & Gimpel, K. (2016). *Gaussian Error Linear Units (GELUs)*. arXiv preprint arXiv:1606.08415.